Beginner Guide to AKS NGINX Ingress Routing

Without an Ingress controller, exposing multiple apps on Azure Kubernetes Service creates expensive cloud load balancers for every single service. NGINX Ingress acts as a single front door that safely routes outside internet traffic to dozens of apps using one public IP.

What you'll learn:

1. 1. Deploying NGINX Controller via Helm
2. 2. Finding Your Azure Public IP
3. 3. Defining Path-Based Routing Rules
4. 4. Troubleshooting 502 and 404 Errors

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

1. Deploying NGINX Controller via Helm

Helm is the package manager for Kubernetes. Running this command pulls the verified NGINX Ingress chart and deploys both the controller pods and an Azure Public Load Balancer. Azure automatically provisions an external public IP address without any manual networking setup in the Azure Portal.

```
$helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
 helm repo update
 helm install app-ingress ingress-nginx/ingress-nginx \
   --create-namespace --namespace ingress-basic \
   --set controller.service.annotations."service\.beta\.kubernetes\.io/azure-load-balancer-health-pro
```

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

2. Finding Your Azure Public IP

Once Helm finishes the deployment, Azure needs roughly 60 seconds to bind a real public IP to your controller service. You must watch the external IP column transition from pending to an actual IP. This single IP will serve as the entry point for all your web services.

```
$kubectl get service --namespace ingress-basic \
   app-ingress-ingress-nginx-controller --watch
```

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

3. Defining Path-Based Routing Rules

An Ingress resource tells NGINX how to distribute incoming requests. In this manifest, any request arriving with the prefix /orders is forwarded to your orders-service running on port 80 inside the cluster. Notice that ingressClassName must match nginx so the controller claims this rule.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress-rules
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$2
spec:
  ingressClassName: nginx
  rules:
  - http:
      paths:
      - path: /orders(/|$)(.*)
        pathType: ImplementationSpecific
        backend:
          service:
            name: orders-service
            port:
              number: 80
```

AKS Ingress with NGINX: Route External Traffic to Your Services

Wednesday, November 11, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

4. Troubleshooting 502 and 404 Errors

If you open your browser and see an NGINX 404 or 502 Bad Gateway page, the ingress controller is healthy but cannot pass traffic to your backend application pod. Stream the controller logs to spot failing endpoints, wrong target ports, or crashing application containers in real-time.

```
$kubectl logs -n ingress-basic \
   -l app.kubernetes.io/name=ingress-nginx \
   --tail=50 -f
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Keep all backend services as ClusterIP to prevent accidental public internet exposure.
+ Always set spec.ingressClassName to nginx in Kubernetes v1.18+ manifests.
+ Use rewrite-target annotations when backend applications do not expect the ingress subpath.
+ Pin your Helm chart versions in production so controller updates do not break unexpectedly.
+ Check pod endpoints using 'kubectl get endpoints <service-name>' when debugging 502 errors.

Watch Out For

! Using LoadBalancer service types for backend apps instead of letting Ingress handle routing, which leads to massive Azure bills.
! Mismatched targetPort in the Service definition, causing NGINX to return 502 Bad Gateway.
! Deploying the Ingress resource in a different namespace than the target application services (Ingress cannot route across namespaces natively).

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh